

SQA-Mashup: A mashup framework for continuous integration



Martin Brandtner*, Emanuel Giger, Harald Gall

University of Zurich, Department of Informatics, Binzmuehlestrasse 14, 8050 Zurich, Switzerland

ARTICLE INFO

Article history:

Received 17 June 2014

Received in revised form 26 September 2014

Accepted 1 October 2014

Available online 28 October 2014

Keywords:

Continuous integration

Controlled user study

Software quality

Tool integration

Information needs

ABSTRACT

Context: Continuous Integration (CI) has become an established best practice of modern software development. Its philosophy of regularly integrating the changes of individual developers with the master code base saves the entire development team from descending into *Integration Hell*, a term coined in the field of extreme programming. In practice, CI is supported by automated tools to cope with this repeated integration of source code through automated builds and testing. One of the main problems, however, is that relevant information about the quality and health of a software system is both scattered across those tools and across multiple views.

Objective: This paper introduces a quality awareness framework for CI-data and its conceptional model used for the data integration and visualization. The framework called SQA-Mashup makes use of the service-based mashup paradigm and integrates information from the entire CI-toolchain into a single service.

Method: The research approach followed in our work consists out of (i) a conceptional model for data integration and visualization, (ii) a prototypical framework implementation based on tool requirements derived from literature, and (iii) a controlled user study to evaluate its usefulness.

Results: The results of the controlled user study showed that SQA-Mashup's single point of access allows users to answer questions regarding the state of a system more quickly (57%) and accurately (21.6%) than with standalone CI-tools.

Conclusions: The SQA-Mashup framework can serve as *one-stop shop* for software quality data monitoring in a software development project. It enables easy access to CI-data which otherwise is not integrated but scattered across multiple CI-tools. Our dynamic visualization approach allows for a tailoring of integrated CI-data according to information needs of different stakeholders such as developers or testers.

© 2014 Elsevier B.V. All rights reserved.

1. Introduction

A fundamental aspect of Continuous Integration (CI) according to Fowler is visibility: “[...] you want to ensure that everyone can easily see the state of the system and the changes that have been made to it.” [1]. The integration of a modern CI-toolchain within the development process of a software project is fully automated, and its execution is triggered after every commit. Developers or testers perceive the CI process most often only in case of a build break or test failure. In such a case, they get an automatically generated notification, for example, via email. A developer can then fix the problem using this information. This kind of exception-driven behavior helps to detect and fix problems as early as possibly during integration runs. Even in the absence of build or test failures, modern CI-tools generate a bulk of report data with each commit.

This data is scattered across the entire CI-toolchain, and analyzing it, for instance, to monitor the quality of a system, is a time consuming task. This can delay the rapid feedback cycles of CI and one of its major benefits is then not utilized.

The need for an integration of the tools used in a CI-landscape is expressed in studies that address the information needs of software developers. Questions are, for example, *What has changed between two builds [and] who has changed it?* [2]. Typically, the way to answer these kind of questions consists of two steps. First, a developer needs to know the dates of the respective builds. Second, these build dates can then be used to investigate, e.g., commit logs, file-diff data, build details, issue details etc. However, to obtain the relevant information, a developer must access several different tools and navigate through multiple views.

Additionally, the scope covered by modern CI-toolchains is not limited to build break and test failures. Quality measurements, such as code metrics, violations of coding guidelines, or potential bugs, are computed with each integration run. They provide an immediate feedback circuit for recently committed source code

* Corresponding author. Tel.: +41 44 635 67 31.

E-mail address: brandtner@ifi.uzh.ch (M. Brandtner).

changes. Currently, this immediate feedback is limited to email notifications from each CI-tool due to the missing integration along a CI-toolchain. Integration approaches, such as the work of Singer et al. [3], address only tools used by developers during the creation of source code.

We derived seven state-of-the-art tool requirements from related approaches and designed and implemented a proof-of-concept mashup for data integration, and a front-end for its representation. Our framework and platform called *SQA-Mashup* is highly extensible and integrates information from various CI-tools, such as BitBucket, GitHub, Jenkins-CI, and SonarQube. The graphical front-end of *SQA-Mashup* is role-based and supports dynamic views associated with different stakeholders in the context of a software project.

We use the term *stakeholder* to refer to developers and testers as these two roles are currently supported. However, we emphasize that our approach can be extended to other roles, such as software architects or project managers. The data presented in a dynamic view can be arranged either automatically or manually, based on the information needs of a stakeholder. For example, the default developer view provides code-related measurements, such as code complexity and others, while the tester view provides testing-related measurements such as test coverage etc. *SQA-Mashup* further supports so-called *timeline views* that use a time axis to visualize time-related information on builds, commits, or test runs. For example, the timeline view can enable the detection of increased commit activities before the daily integration run. The increased commit activity before an integration run can be an indicator that developers, for example, do not regularly check in their source changes, which can lead to error-prone merging scenarios and build breaks.

To validate our approach we defined the following research question:

RQ: How do stakeholders perform in answering questions about software quality with *SQA-Mashup* compared to the use of standalone CI-tools?

To answer this research question we conducted a user study with 16 participants on the JUnit project. The results show that an integrated platform has substantial advantages: information needs covered by an integration of CI-tools lead to more accurate answers in less time compared to the standalone use of CI-tools.

This work is an extension of our research presented in [4], which introduced the *SQA-Mashup* integration approach. On top of this earlier work, we newly introduce the *SQA-Mashup* framework and its conceptional model used for the data integration and visualization.

The remainder of this paper is structured as follows: In Section 2 we introduce the *SQA-Mashup* approach. We describe our *SQA-Mashup* framework and its a proof-of-concept implementation in Section 3. The experimental design of our user study is described in Section 4. In Section 5 we present our main results and discuss our insights for information needs and tooling. We discuss related work in Section 6 and conclude with our findings in Section 7.

2. Mashing-up software quality data

The goal of our approach is to integrate the information scattered across diverse CI-tools into a single Web-based service. The information is presented according to the information needs of different stakeholders. We aim for a fast and easy way to analyze and communicate the actual state, the current health, and the most recent changes of a system. Fig. 1 depicts the integration of data from a CI-toolchain to present it to different stakeholders. The *CI-toolchain – Tool view* in Fig. 1 represents the information

gathering process of, for example, a software tester during a quality measurement review. Starting from a quality analysis platform (A1), where the tester detects an increase of convention violations, the developer navigates to the build platform (A2) and from there to the associated source code commits (A3) to locate the causes of the violations. The data accessed during such an information gathering process can be modeled as a graph as depicted in the *CI-toolchain – Graph model*. This graph models each CI-tool as a tree with nodes representing the different data available in a tool. However, during a development or testing task, such as bug fixing or a quality review, only some data of each CI-tool is relevant to solve the task. The highlighted nodes (D1–D5) in the tree model indicate the data accessed by the software tester during his review. Our proposed mashup integrates the single pieces of data from different CI-tools into a condensed model (*SQA-Mashup – Graph model*). It is used to dynamically represent (*SQA-Mashup – Tool view*) the data according to the information needs of a stakeholder (see Section 2.4).

In the following, we discuss our way of data integration and presentation. We present our mashup-based data integration that is inspired by Yahoo pipes.¹ In Sections 2.4 and 2.5 we present the concepts of dynamic and timeline views.

2.1. Requirements

Our integration approach builds on a mashup-based paradigm with focus on data originating from CI-tools, which have to fulfill the following requirements:

- RESTful Web-service API (with media type JSON)
- Unique project identifier (e.g., maven artifact, etc.)

In modern CI-tools, the initial access to the data is facilitated by means of Web-Service interfaces. Our approach builds on these Web-service APIs and supports CI-tools with RESTful Web-service interface only. The integration of APIs not based on a RESTful Web-service is possible, but requires an adequate wrapper acting as Web-service.

In addition, it is necessary that each CI-tool provides a unique project identifier to enable an accurate interlinking of data across the CI-tools. An example for such a project identifier is the *artifact group* and *artifact name* when Maven is used as build automation system in a software project. Within an integration scenario it is not necessary that each CI-tool provides the same unique project identifier as a mapping between identifiers is supported by our approach.

2.2. CI-tool integration

We address the integration and interlinking of data across various CI-tools with a *mashup*, which enables a flexible selection of data from multiple data sources and to integrate them into a single consistent information space. We define so-called *integration pipes* to describe the processing steps required to interlink data from Web-services of different CI-tools. Fig. 2 depicts an example integration pipe, which (1) collects data from Jenkins-CI and SonarQube, (2) aggregates the data, and (3) prepares the data for the visual representation in a *widget*.

The numbered arrows in Fig. 2 illustrate the execution of an integration pipe: the execution is triggered via a request of the according Web-service interface provided by the proposed mashup framework. We use the term *document* to address data structures, which can be used as input or output of a processing step. In this

¹ <http://pipes.yahoo.com/pipes/>.

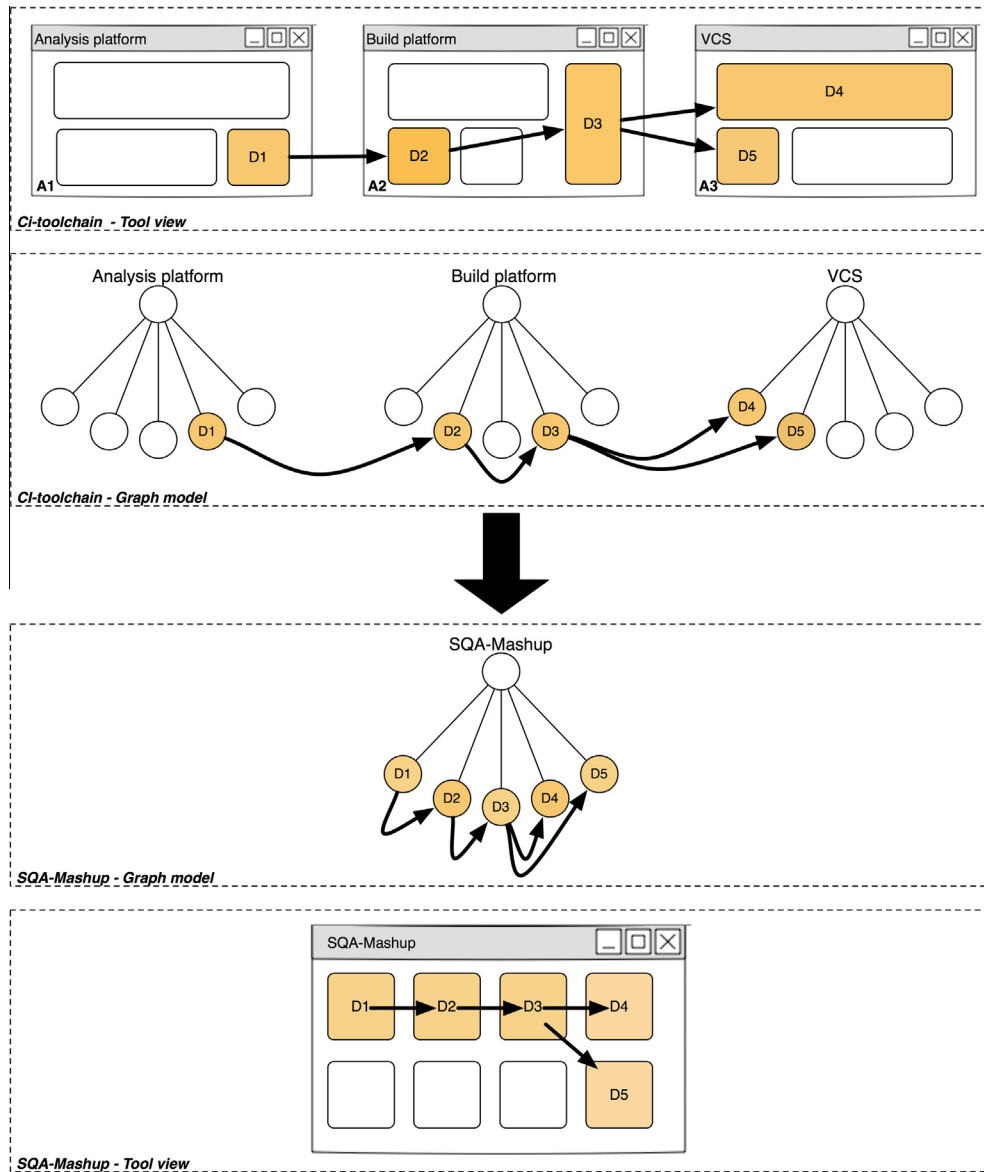


Fig. 1. SQA-Mashup approach: Integration overview.

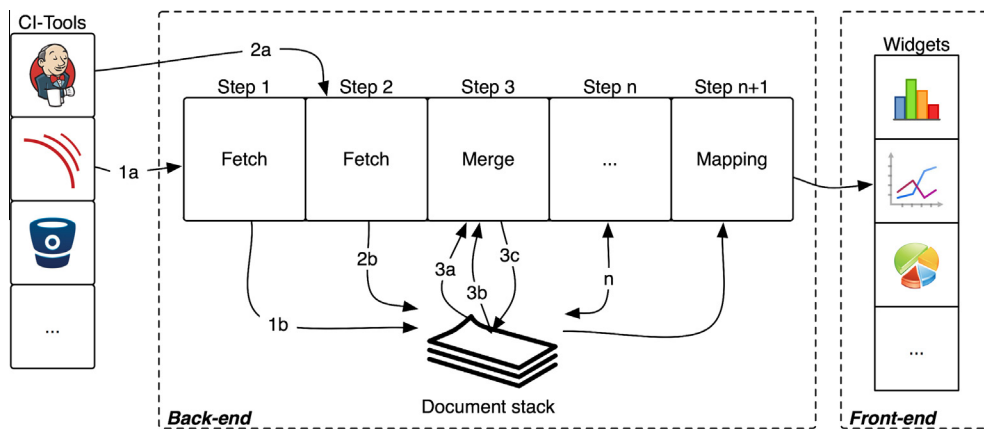


Fig. 2. SQA-Mashup - Pipe-and-filter-based data integration.

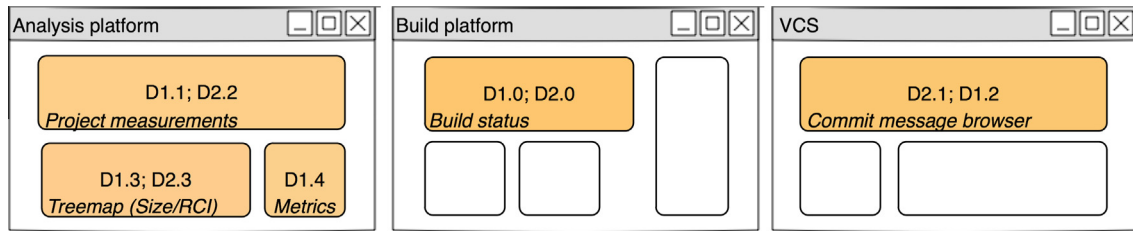


Fig. 3. CI-toolchain – Data accessed by developer D1 and D2.

exemplary integration pipe, the execution process fetches data from a SonarQube Web-service (1a) and stores it to the *document stack* (1b) of the executor thread. The document stack is used to transfer data between the independent processing steps of an integration pipe. In the second processing step, the executor fetches data from a Jenkins-CI Web-service (2a) and pushes it on top of the document stack (2b). The third processing step of this integration pipe is a so-called *merge* step. The merge pulls two documents from the document stack (3a, 3b), merges them by concatenation, and stores the resulting document to the document stack (3c). Theoretically, n further processing steps can be part of such an integration pipe, as indicated by *Step n*. The last step (*Step n + 1*) pulls the remaining document from the stack, maps the retrieved data into an adequate format for the front-end, and sends the resulting document to the client, which has triggered the pipe execution.

2.3. Data processing

The SQA-Mashup approach builds on the JavaScript Object Notation (JSON) as a flexible data exchange format. A standard stack approach is used for the data forwarding between the single integration steps. The combination of a flexible data exchange format and a generic document stack enables a clean encapsulation of the single processing steps.

The data processing steps that have to be supported by our SQA-Mashup approach are listed below. However, this is just an initial set of processing steps, which can be extended in future work.

- **Fetch:** The Fetch queries data from Web-services and stores it on the document stack. The expected parameters for this processing step are a URL to the Web-service and a name of an appropriate Web-service endpoint handler. An endpoint handler takes care of connection management tasks, such as authentication during Web-service queries.
- **Merge:** The Merge pulls two or more documents from the document stack, merges them, and pushes the resulting document back to the stack. The expected input parameters are two or more value-pairs. Each value-pair describes the mapping of an attribute from a source document to the resulting document with an XPath expression.
- **Mapping:** The Mapping integrates the aggregated data from different CI-tools and maps it in an adequate format for the visual representation in a *widget* (see Section 2.4).

An processing step that is currently not supported explicitly by SQA-Mashup is the *Transformation* of documents. For example, the use of the data fetched from SonarQube (Fig. 2 – 1a) as an input to query data from Jenkins-CI (Fig. 2 – 2a). In such a case, it is necessary to transform the response of SonarQube to match the input format of the Jenkins-CI query interface. At the current stage, the transformation step is part of the Fetch and the Mapping step. The Mapping as-well as the Fetch step can use documents from

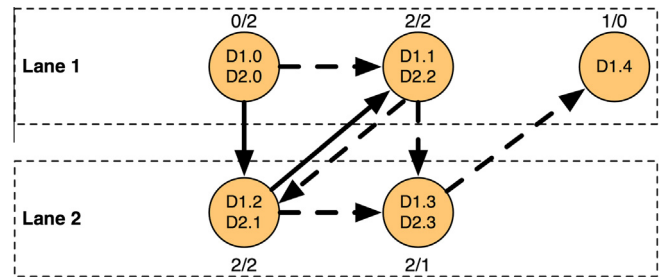


Fig. 4. Graph – Data nodes accessed by developer D1 (---) and D2 (—) inclusive number of edges (in/out).

the document stack as input data. The data transformation can be parameterized within the according step with XPath expressions.

2.4. Dynamic views

The visual representation of our SQA-Mashup approach builds on a concept called *dynamic views*. Dynamic views allow for an integration of data from different CI-tools into screens addressing the information needs of different roles, such as developers and testers. The concept of a dynamic view is built on a graph-based dependency model for CI-data and information needs. For example, Fig. 3 depicts Web-browser windows with CI-tools and highlights relevant data for satisfying two independent information needs of two developers (D1, D2). In addition to the information about the accessed data, the sequence of the data accessing flow is given by the sub-category number (e.g., Dx.1, Dx.2, etc.). Based in this information we can generate a graph to model dependencies between different data from multiple CI-tools. Fig. 4 shows such a graph, which represents the data access flow of the scenario introduced in Fig. 3. The nodes in the graph are already arranged based on the number of dependencies for both data access flows (D1.x and D2.x). We used the valency of the nodes to express the importance of data in the information gathering process. A node with a high valency tends to be closer to the starting node compared to nodes with a small valency. In this example, we want to map the values of the graph into a view of $2 \times m$ widgets. A widget in our context is a graphical representation of the output of an integration pipe and can be a chart, a list or any other visualization. Based on the $2 \times m$ widget matrix, we restricted the space to visualize the nodes to two lanes. The node with zero inbound edges (D1.0/D2.0) in Fig. 4 was chosen as starting point for our example. It represents the build status on which many other CI-tools, such as quality analysis platforms, depend on for the further processing of an integration run. Based on the valency, the nodes D1.1/D2.2 and D1.2/D2.1 have the same importance for a developer and get accessed right after or in the second step after the build status. In our example, node D1.1/D2.2 represents an information grid with various project measurements and node D1.2/D2.1 represents a commit message browser. The data behind node D1.3/D2.3 is

accessed by both developers and it is the exit point for the information gathering process of developer D2. In our example, node D1.3/D2.3 represents a tree map with information about the package size and the rule compliance. Developer D1 additionally gets information from node D1.4, which in our example is a detailed list of code size measurements.

2.5. Timeline view

The *timeline view* builds on a concept that is complementary to the dynamic view concept. CI-tools generate events (e.g., build failures), which are presented as status feeds directly in the user interface or as Rich Site Summary (RSS) feeds. Fig. 5 depicts CI-tools with example events published over independent RSS feeds. Based on such a feed, a stakeholder can keep track of what is going in a software project. The drawbacks of such feeds are the missing integration across multiple CI-tools and the missing presentation of time-related dependencies. For example, a feed of the VCS may contain the last ten commits of the current day whereas the feed of the build platform may contain the last ten builds of the last week. The feed with commits is far ahead of the feed with the build

information because the frequency in which commit events occur is much higher. In such a case, a stakeholder has to imagine a time line before it is possible to interpret the information of both feeds. Fig. 6 shows a visualization approach that provides a slightly different view on such events fired by CI-tools. The major difference to a simple feed is the representation of the timely correlation between the single events, which can help a stakeholder to better and faster understand the current situation in a software project. An integration along a time axis may foster the understanding but the view can still become overcrowded. For example, commit events occur in higher frequency than events of other CI-tools, which leads depending on the scale (see Fig. 6) to an overlapping in the visualization. To address the issue of an overcrowded screen it becomes necessary to provide a filter mechanism. Fig. 7 depicts a conceptional approach to highlight certain information and filter out less important ones. The filter mechanism has to support a filtering on event attribute level to enable a selective filtering across multiple CI-tools as illustrated in Fig. 7. For example, a stakeholder wants to locate all commits without a meaningful commit message based on a dictionary containing words such as *bug-fix*, *no comment*, *small fix*, etc. Such a commit message filter combined with

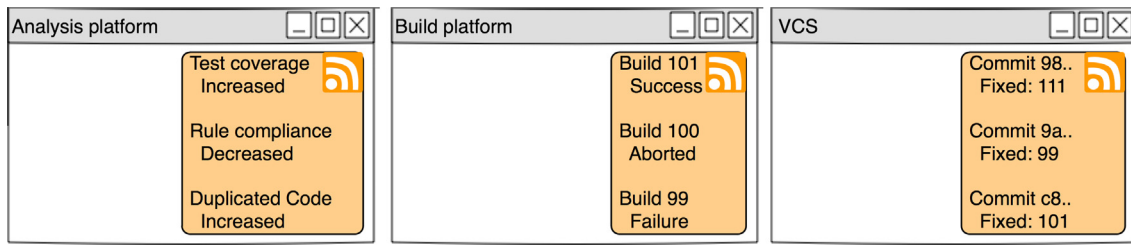


Fig. 5. CI-toolchain – Event feeds.

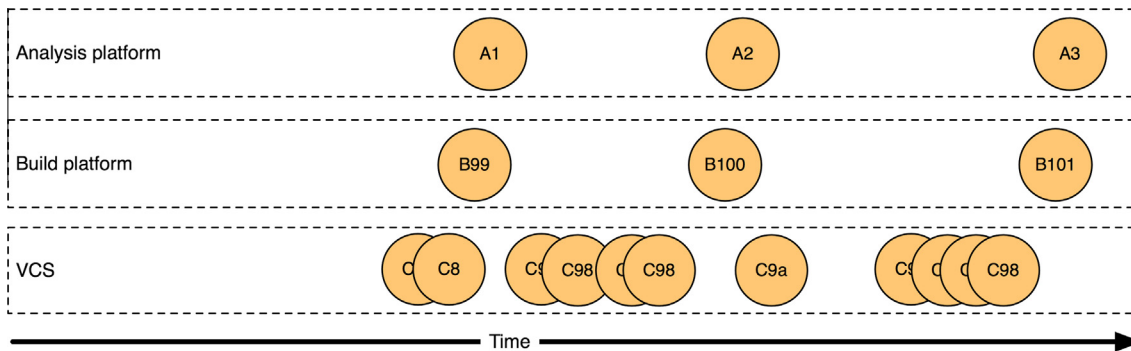


Fig. 6. CI-toolchain – Event timeline.

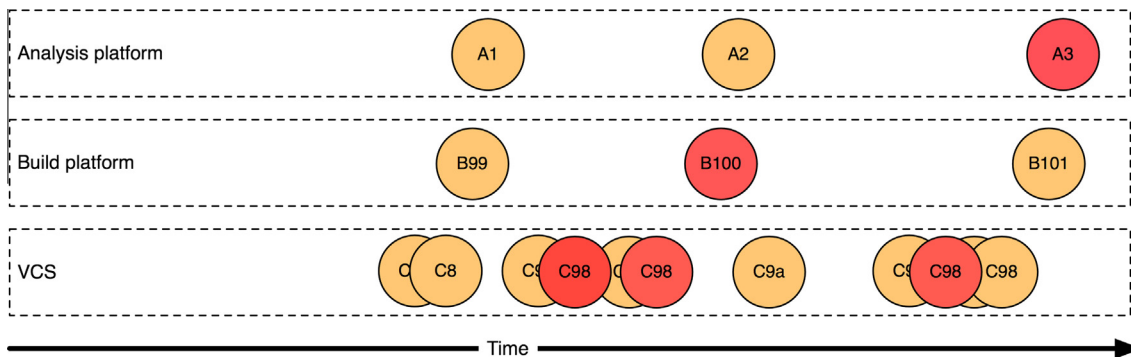


Fig. 7. CI-toolchain – Event filter.

a filter on failed builds can support a developer to faster locate commits without meaningful message and their influence on build failures.

3. SQA-Mashup framework

In this section, we describe a proof-of-concept implementation of the SQA-Mashup framework. A service back-end integrates CI-data through integration pipes for a dynamic user interface composition (see Section 3.2) based on information needs. We leverage this flexibility for a dynamic visual presentation of aggregated CI-data through widgets (see Section 3.3). Every integration pipe of the SQA-Mashup framework back-end can be visualized through a *widget* in the front-end. A strict separation of concerns enables the visualization of one pipe output with different widgets, such as a list, a chart, or a tree-map. We pushed this flexibility even further and enabled the creation of *views*. A view is an individual arrangement of widgets according to the needs of a stakeholder. In the current implementation, we offer a view for developers and a view for testers. These two views differ in the kind and the granularity of the presented information. For example, the pipe with data about the unit tests is presented in the developer view as a chart of passed and failed test cases; in the tester view the pipe is presented as a list with more detailed information.

In the following, we discuss our proof-of-concept implementation of the SQA-Mashup approach. We discuss tool requirements and reflect the associated literature. We then present our proof-of-concept implementation of the SQA-Mashup back-end followed by a brief description of its flexible Web-service API. In Sections 3.3 and 3.4 we present a proof-of-concept implementation of the SQA-Mashup front-end.

3.1. Tool requirements

We surveyed the literature for tool requirements of software engineering tools. The work of Mockus et al. [5] and LaToza et al. [6] was chosen as starting point for our survey. We investigated related work and looked for explicitly-stated and implicitly-stated tool requirements.

Table 1 lists requirements for the integration platform presented in this paper. The column *kind* indicates if a tool requirement is explicitly (E) stated in literature or can be derived (D). We categorized each tool requirement based on the context in which it was stated: change awareness (CA) and tool-design-related requirements, such as user interface (UI) and architectural design (AD).

The entries in Table 1 show that more recent literature has a strong focus on change awareness, whereas traditionally, the focus has been on architectural design and user interfaces. The reason for this can be found in recent technological innovations, from which service-based architectures emerged. Nowadays, many CI-platforms offer a Web-service based interface to exchange data with other systems. Tool requirement R6 reflects this functionality and requests for interoperability rather than a simple data exchange

over pre-defined interfaces. Interoperability has to guarantee that exchanged data can be processed in each of the associated systems. Current CI-tools exhibit limited interoperability, because their service interfaces are a collection of flat data optimized for the associated user interface. A tight integration between service interface and user interface is contradictory to R7. The Web-service interface of Jenkins-CI is an example for a Web-service optimized for the default front-end. Independent development of a user interface is possible if the data layer is designed in a generic way or offers configurable services. Our definition of configurable services is described in Section 3.2.

A dynamic arrangement of UI-elements in state-of-the-art tools is possible, but often restricted to one configuration per project. Personalized views according to a stakeholder role and personal preferences are not supported by CI-tools, such as SonarQube. It is possible to address this issue with plugins, for example, to achieve a simple rearrangement of widgets for a personalized view. However, the capability to combine widgets from multiple views within one view is often restricted by the underlying architecture of a CI-tool. For example, a tool has one view on development data and a second view on testing data. The ability of a CI-tool to handle combined views is stated in R2. A similar tool requirement, but yet on another level, is stated through R5. The dynamic level added through R5 allows for interactive views based on a stakeholder's role. An interactive visualization adopts the view of a stakeholder based on latest changes in the data of a CI-toolchain and with respect to the affected stakeholder's role.

Agile development processes do iterations within the development and maintenance phase, compared to other software development processes. More iterations within the same amount of time require the ability to respond faster to changes happening during an iteration. CI-tools support agile software development, but each single tool restricts the data presentation to its own data. Within each iteration, a stakeholder has to navigate through every tool to become aware of changes since the last build. Tool requirement R4 addresses the awareness of changes within each build. A similar direction is stated by tool requirement R3. The difference between tool requirement R3 and R4 is the point of view: R3 addresses the awareness between co-workers, whereas R4 addresses the awareness of changes within a certain period (for example, an iteration). R1 has a special focus on software quality measurements. Similar to R4, the aim of R1 is to become aware of changes.

3.2. Web-service presentation

The Web-service interface design plays an important role for satisfying tool requirement R7. The design of a Web-service interface aims for a clear separation of concerns and the avoidance of redundant values in response data. This is different to a user interface that provides redundant data to allow a stakeholder to easily discover dependencies between data from different CI-tools based on the tool requirements R1–R4.

Table 1
Tool requirements.

ID	Tool requirement	Kind	Cat	Ref
R1	Be able to locate quality hot-spots in source code (e.g. low test coverage, high complexity)	D	CA	[6]
R2	Permit dynamic arrangement of information shown in the user interface	D	UI	[2]
R3	Provide awareness of the activities of peers (co-workers)	D	CA	[7]
R4	Be able to discover immediately where changes occurred, when they were made, and who made them	E	CA	[5]
R5	Provide an interactive visualization of a person's role (e.g., developer, tester, manager)	E	UI	[5]
R6	Be able to interoperate with other software engineering tools	E	AD	[3]
R7	Permit the independent development of user interfaces (clients)	E	AD	[3]

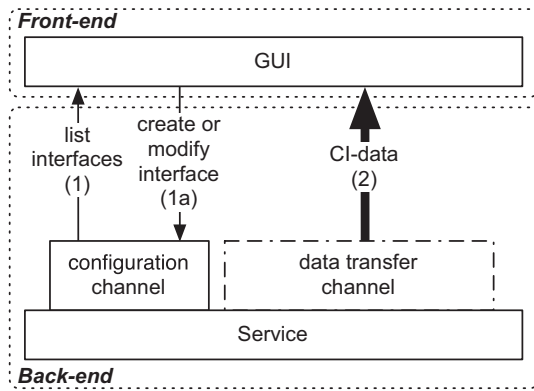


Fig. 8. Dynamic Web-service definition through channels.

We overcome the divergence between the Web-service interface description and the visual representation with a client–server architecture and a specific design of the Web-service interface (Fig. 8). The Web-service design consists out of a *configuration channel* and a *data transfer channel* for the communication between the service and a client.

The data transfer channel contains Web-services for the actual transfer of data from CI-tools to a client application, such as a graphical user interface (GUI). Each Web-service in the data transfer channel can be dynamically created, modified, or deleted during runtime. The logic to manage the data transfer channel is encapsulated in Web-services, which are part of the configuration channel. All Web-services used in the configuration channel are hard-coded and cannot be modified during runtime. The Web-services of the configuration channel represent an API which is the

starting point for a client application. Every client application with access to this API is able to consume, adopt, and extend the data transfer channel. Such a Web-service design enables an encapsulation of the complete integration logic and computation efforts into one place. This is especially interesting for concerns on change awareness, such as tool requirements R2 and R4. An example is an application which visualizes the evolution of a software project based on data from CI-tools. Depending on a software project's history the amount of evolution data can be huge. For such a large amount of data our approach enables a visualization client to query meta data, which can be used to limit the actual query size. Smaller response values which accord to the needed data format save computation power on the client device. The saved computation power can be used for enhancements such as richer or faster visualization.

We use Representational State Transfer (RESTful) Web-services in combination with the JavaScript Object Notation (JSON) for the data collection, aggregation, processing, and propagation. JSON is a text-based data-exchange format with a low processing overhead and, in combination with RESTful Web-services, a de facto standard in CI-tools.

3.3. Developer and tester view

The developer and tester view of the SQA-Mashup front-end is built on the graph concept introduced in Section 2.4. Fig. 9 depicts a screenshot of a role-based view with information for a software developer representing the graph nodes shown in Fig. 4 as widgets. The figure shows different widget types and data from different information sources. For example, the *Infogrid* widget contains information about the testing (e.g., unit tests success rate) and the source code quality (e.g., rule compliance) of the SQA-Mashup project. The *Rules Compliance* widget shows a tree-map with

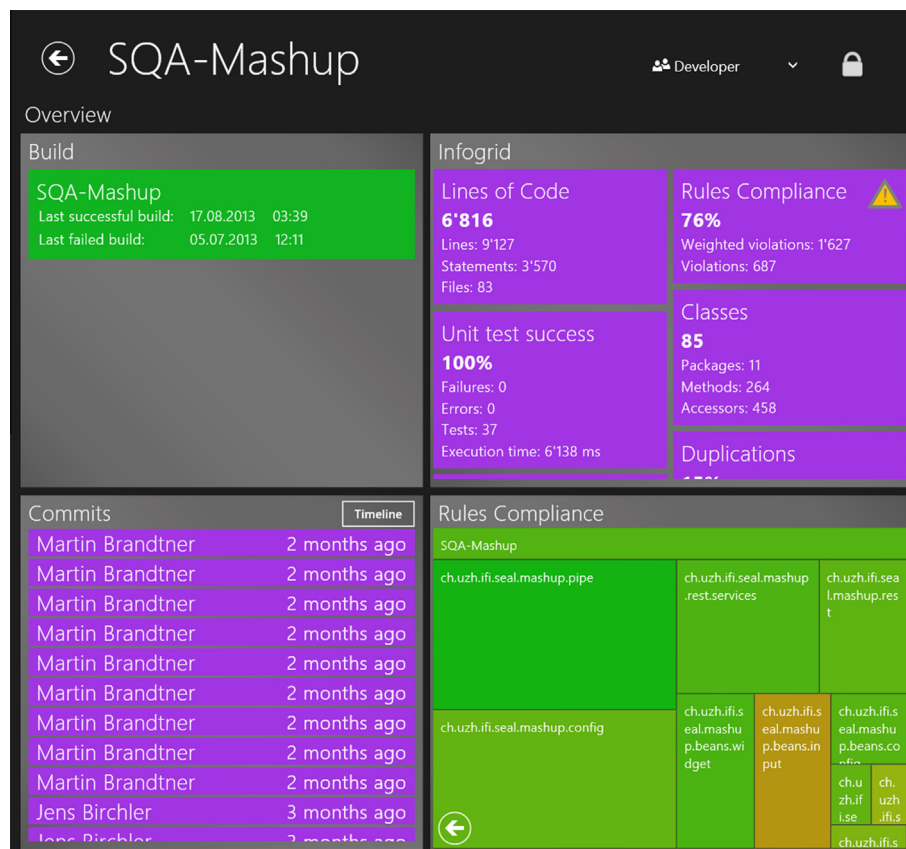


Fig. 9. SQA-Mashup – Developer view.

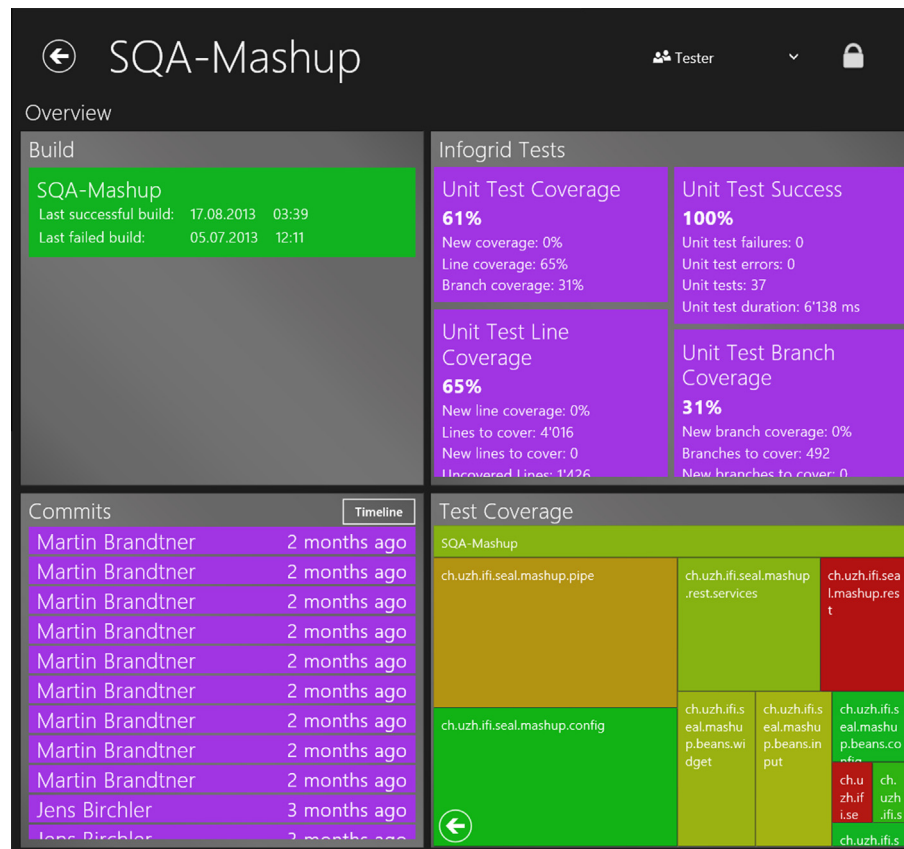


Fig. 10. SQA-Mashup – Tester view.

information about the package size (rectangle size) and the rule compliance of each package (rectangle color) in the project under inspection. SQA-Mashup currently supports several well-established visualizations in the respective widgets: bar charts, line charts, pie charts, spider charts, tree maps, and lists. It is possible to extend SQA-Mashup with further and more specific widgets to visualize data from CI-tools. Furthermore, the approach of SQA-Mashup is not restricted to role-based views. It is possible to create, for example, individual views for a stakeholder as well.

The tester view in Fig. 10 has a similar appearance as the developer view in Fig. 9 but is yet different in meaning. For example, the *Test Coverage* widget looks similar to the *Rules Compliance* widget in the developer view. However, the color indicates the test coverage in the tester view instead of the rule compliance in the developer view. This is one example how dynamic views can satisfy different information needs by presenting various perspectives onto the integrated data with a similar visual appearance.

The arrangement of the widgets in a view can be done automatically via a Web-service interface or manually by drag-and-drop of single widgets into the dynamic view. In the developer and tester view the arrangement of the widgets is chosen according to the granularity and dependency of the presented information. For example, both views have the build status from Jenkins-CI and the commit data from the VCS in the first column. These data can be queried independently from the according CI-tools. The data presented in the second column can be queried independently as well but it is associated with the data of the first column. For example, there are no coding convention checks and unit test executions in case of a build failure because quality analysis platforms, such as SonarQube, work only with compilable source code. In case of a build failure, our approach presents the rule compliance and test coverage results of the last successful build, which is the same

functionality as in state-of-the-art CI-tools. In contrast to state-of-the-art CI-tools, a stakeholder can immediately see whether or not the test coverage or rule compliance data is up-to-date in SQA-Mashup because of the build status information, which is visualized next to the quality data.

The accuracy of quality data is essential for the work of developers, testers, and for the project management. Especially in cases when a build failure retains over a longer time period the actual test coverage may deviate from the indicated one, which was calculated during the last successful build. Therefore, it has to be assured that a stakeholder can easily verify if the data is up-to-date.

3.4. Timeline view

The timeline concept introduced in Section 2.5 is implemented in the SQA-Mashup front-end as depicted in Fig. 11. It integrates event data from a build tool (green²/red), an issue tracker (blue), and a VCS (purple). The yellow bar on the right side indicates that this issue is currently selected and the detail information is shown in the bottom area of the view.

The timeline view enables a stakeholder to navigate through the history of a software project and to directly access detailed commit, issue and build data from multiple CI-tools in one screen. It is possible to filter events based on the CI-tool in order not to overwhelm a stakeholder with a large amount of data. Furthermore, a more fine-grained filtering based on the metadata of events is implemented as-well. For example, it is possible to show only commit messages which do not contain a commit message or a generic message, such as a *bug-fix*.

² For interpretation of color in Fig. 11, the reader is referred to the web version of this article.

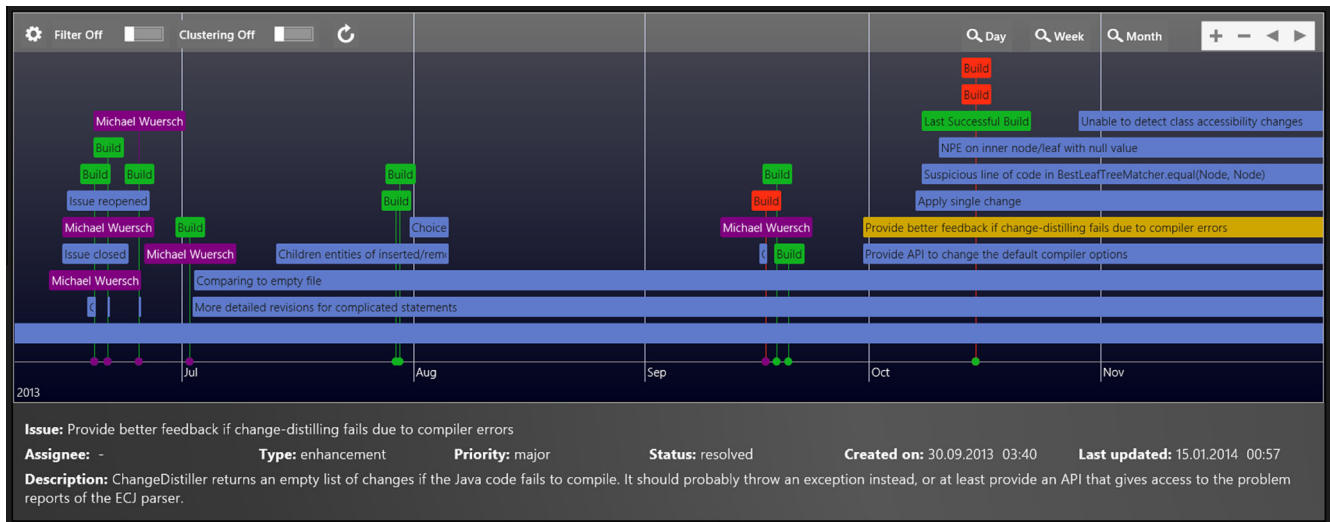


Fig. 11. SQA-Mashup – Timeline view.

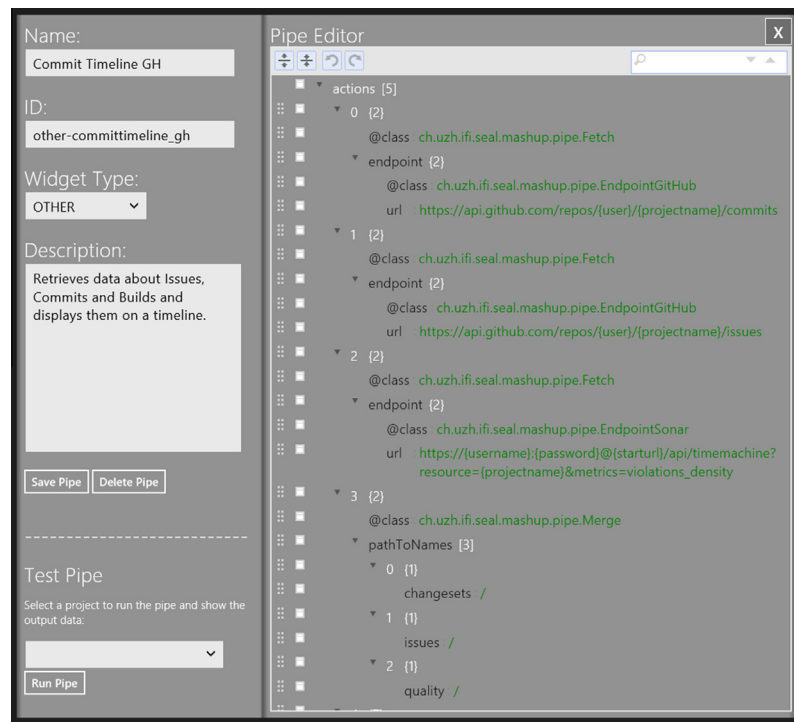


Fig. 12. SQA-Mashup – Integration pipe configuration.

The data visualized in the timeline view has a different focus compared to the dynamic view. At the current stage, the dynamic views are used to visualize a snapshot of the data of the last integration, whereas the data presented in the timeline view focuses on the development history of a software project. Of course, it is still possible to navigate to the snapshot date to see the latest events which occurred in a CI-toolchain.

The timeline view is an additional perspective to navigate through the history of a software project to better understand the temporal relation of data presented in the dynamic views. A combination of the timeline view and the dynamic views may help a stakeholder to answer information need questions, which are hard to answer with state-of-the-art CI-tools (e.g., [6]).

3.5. Integration pipe configuration

In our approach, a configuration of an integration pipe is project independent and only associated to the used CI-tools (e.g. Jenkins-CI, etc.). Once an integration pipe is defined it can be used for different software projects in the same CI-toolchain. The pipe configuration of SQA-Mashup depicted in Fig. 12 builds on the pipe-and-filter concept introduced in Section 2.2.

In Fig. 12 the configuration of the *Commit Timeline GH* integration pipe is shown in the configuration editor. The *Pipe Editor* depicts a visual representation of a raw integration pipe configuration with four integration steps. The kind of widget used to visualize the data output of the according pipe (e.g., bar chart, etc.) is

defined in the field *Widget Type*. A function called *Test Pipe* allows for a test run of a pipe on a software project, which is registered in SQA-Mashup to validate the integration process.

The current stage of the configuration screen is intended to be used only by professional stakeholders, which are familiar with the Web-services of the corresponding CI-tools. There is, for example, no wizard which supports the automatic joining of data from different CI-tools. A stakeholder has to manually select the fields used to join the data of two or more documents from the document stack. Nevertheless, it is possible to automate the composition of new integration pipes by utilizing the Web-service interface provided by the SQA-Mashup back-end.

SQA-Mashup is available in the Windows 8 App Store³ and comes with a set of integration pipes to aggregate data from the CI-tools, which were used in our controlled user study (see Section 4). Many of these integration pipes contain not more than the two integration steps *fetch* data and *map* data into a widget. The simplicity of these integration pipes is given by the fact that the data provided by Web-services of CI-tools are close to the CI-tool's user interface. For example, the data of the overview screen in the Jenkins-CI user interface can be fetched with one Web-service call. The largest integration pipes in SQA-Mashup are the pipes used for the data aggregation of the timeline view. In such a pipe, the event data of all CI-tools gets fetched and integrated for the visualization in the front-end.

4. Controlled user study

We designed and conducted a controlled user study with 16 participants who had to solve nine software maintenance tasks. The participants were randomly assigned to either the control group or the experimental group. Each group consisted out of eight subjects and the control group confined the baseline of our study. This group had to use state-of-the-art CI-tools and a social coding platform: Jenkins-CI and SonarQube as the CI-tools and GitHub as the social coding platform. The experimental group had to use SQA-Mashup, which was implemented according to the information needs listed in Table 1. More information about the study population is provided in Section 4.2 and the nine tasks are listed in Table 3.

We formulated three hypotheses (see Table 2) and verified them through statistical tests based on the results of the user study. The hypotheses address the total values of the score, the time, and the system usability score. The total score and time are summed up over all tasks per participant.

4.1. Tasks – Nine questions from the literature

We exclusively selected tasks out of the software maintenance and evolution field which have been used in other studies or information need questions stated in literature. This is to ensure an objective comparison of our SQA-Mashup approach and the baseline approach with CI-tools.

We composed nine tasks which can be categorized along the domains *Program Comprehension*, *Testing*, and *Cross-Domain* questions. The tasks originate from the work of Aranda et al. [7], Fritz and Murphy [2], LaToza and Myers [6], Mockus et al. [8], and Wetzel et al. [9].

Our aim was to preserve each task in its original structure and semantics. We applied a two-step procedure to achieve this aim. The first step was to replace all general expressions in a task with a specific entity of the software maintenance and evolution domain. For example, the question “*Is this tested?*” was refined to

Table 2
Hypotheses.

ID	Null hypothesis
H1 ₀	There is no difference in the total score per subjects between the experimental and the control group
H2 ₀	There is no difference in the total time per subjects between the experimental and the control group
H3 ₀	There is no difference in the total system usability score between the experimental and the control group

“*Is Class_a tested?*”. In a second step we replaced placeholders such as *Class_a* with entities from the software project used for the user study, in our case the *JUnit* project. We provide a short statement for each task to outline the reason why we selected it.

Table 3 lists the nine tasks with a short statement, categorized by their nature, and with references to the literature.

4.2. Study setting

We ran the user study with 16 participants (S1-S16). Each subject had at least five years of development experience in general and minimum three years development experience with Java. Table 4 lists the degree level, the age, and the overall developer experience of all participants. Furthermore the column *Job* indicates if a participant works part-time in industry. 14 participants were advanced Master students in Computer Science. 11 out of these 14 students worked part-time in industry. The remaining two participants were researchers on PhD level. The students were recruited from courses, which were held by people other than the main authors. The control group and the experimental group consist of eight participants each. Every subject was randomly assigned to either one of both.

Our SQA-Mashup approach allows for an integration of data that originate from a CI-toolchain. The tools used in a CI-chain can vary between software projects. As the baseline of our study, we decided to stay with two major open source developer communities and selected the CI-tools used in the Eclipse and Apache community. The Apache Software Foundation (ASF) provides Jenkins-CI and GitHub repositories for all projects under the ASF Development Process. SonarQube provides an instance⁴ which regularly performs quality analysis of Apache projects. The Eclipse Foundation provides Hudson-CI and GitHub repositories for all projects under the Eclipse Development Process. Therefore, we decided to use Jenkins-CI, GitHub, and SonarQube as baseline CI-tools in the control group of our study. Hudson-CI and Jenkins-CI are similar in their usage as they are forks of the same codebase. The experimental group of our study had to use the SQA-Mashup front-end. The input data was gained from the same CI-tools instances used by the control group.

Each participant of our study had to solve nine tasks (described in Section 4.1) in the context of the JUnit project hosted on GitHub. JUnit [10] is a popular unit testing framework for source code written in Java. The project is under active development since a decade. The JUnit project team uses the built-in issue tracker of GitHub to manage bugs and feature requests. At the time of our study the JUnit project on GitHub had in total 1650 revisions, about 30 k lines of code, 54 contributors, and 736 issues. We decided to select the JUnit project because of the mature stage of the project. At the time of our study the commits indicated that most of the source code changes address bugs or refactoring tasks. Only a small set of commits introduced new features. We think it is therefore a

³ <http://www.ifi.uzh.ch/seal/research/tools/sqa-mashup>.

⁴ <http://nemo.sonarqube.org/>.

Table 3

Tasks.

ID	Task	Ref
<i>Program comprehension domain</i>		
T1	Description. How big is the source code of <i>Project_a</i> ? Statement. Values such as lines of code, number of classes, and number of packages allow a first approximation of the project size and structure	[6]
T2	Description. The three classes with the highest method complexity in <i>Project_a</i> ? Statement. The refactoring of complex parts of source code leads to a better maintainability of software	[9]
T3	Description. What is the evolution of the source code in <i>Project_a</i> ? Statement. To understand a software, it is important to know about the evolution. One way to describe the evolution of a software is to look at testing coverage and coding violations over time	[2]
<i>Testing domain</i>		
T4	Description. Is <i>Class_a</i> tested? Statement. Untested parts of source code does potentially increase the risk of bugs	[6]
T5	Description. Which part of the code in <i>Project_a</i> takes most of the execution time? Statement. The performance of a system can be influenced by a single piece of code. The execution time of unit test can be used to find such pieces with a weak performance	[6]
T6	Description. Which three packages have a test coverage lower than the overall test coverage in <i>Project_a</i> ? Statement. “Code coverage is a sensible and practical measure of test effectiveness.” [8]	[8]
<i>Cross-domain</i>		
T7	Description. What are the builds between the status of <i>Issue_a</i> has changed? Statement. The status changes of an issue can be used to track recurring bugs with each build	[7]
T8	Description. What have my coworkers been doing between <i>Date_a</i> and <i>Date_b</i> ? Statement. Quickly assessing the current status in a software project can support context switches between multiple projects	[2]
T9	Description. What has changed between <i>Build_c</i> and <i>Build_d</i> ? Who has changed it? Statement. Finding changes which introduced a bug but still let the source code compile can cause substantial effort	[2]

Table 4

Study participants.

Subject	Grp	Deg. level	Age (y)	Dev. exp. (y)	Job
S1	E	PhD (CS)	32	8	N
S2	E	PhD (CS)	27	7	N
S3	E	MSc (CS)	27	6	Y
S4	E	MSc (CS)	40	5	Y
S5	C	MSc (CS)	24	5	Y
S6	C	MSc (CS)	26	10	Y
S7	C	MSc (CS)	24	5	Y
S8	E	MSc (CS)	24	5	N
S9	E	MSc (CS)	29	8	Y
S10	C	MSc (CS)	25	5	Y
S11	C	MSc (CS)	26	5	N
S12	C	MSc (CS)	26	5	N
S13	E	MSc (CS)	34	5	Y
S14	E	MSc (CS)	25	6	Y
S15	C	MSc (CS)	27	8	Y
S16	C	MSc (CS)	22	5	Y
Median	C		25.5	5.0	
Median	E		28.0	6.0	
SD	C		1.6	1.9	
SD	E		5.3	1.3	

good example for a software project which is in its maintenance phase.

The maximum time to work through all tasks of our study was 45 min with maximum of five minutes per task. After five minutes a participant had to stop to work and to go on with the next question. The execution of one study in total lasts approximate 40–60 min as there was no time restriction on the introduction and the feedback section.

4.3. Usability

We were particularly interested in the usability of the CI-tools when the study participants solved the tasks. We decided to use the widely-known *System Usability Scale* (SUS) by Brooke [11]. The SUS is a simple and easy understandable scheme which consists out of ten questions each rated on a scale from “Strongly

Agree” to “Strongly Disagree.” The subjects of the control group were asked to give a cumulative feedback for all CI-tools they used during the study and the experimental group rated SQA-Mashup.

As part of our user study each participant was asked to rate the *Difficulty* of each task right after solving it. We used a scale from “Very difficult” to “Very easy” in which a participant provides a personal opinion. This additional information should allow us to sharpen our understanding of the usability rating.

4.4. Performing the study

We performed the user study in four sessions in the lab of our research group in Zurich. The setup for the experimental group was a computer with Windows 8 and the SQA-Mashup front-end installed on it. The setup for the control group was a standard Web-browser such as Mozilla Firefox or Google Chrome. The subjects of the experimental group started at the project page of JUnit in the SQA-Mashup front-end. Participants of the control group started with three Web-browser windows. One window for Jenkins-CI, one for SonarQube, and one for the project page on GitHub.

At the beginning of every session we explained the structure of the study in addition to the written explanation on the questionnaire. The participants solved the tasks independently and the time restriction was enforced with individual stopwatches. We asked the participants of the user study to do a self-assessment of their Programming skills as-well-as their working experience with the CI-tools used in the study. Fig. 13 depicts the self-assessment results of the control group and the experimental group. Only one participant had no working experience with any of the used platforms such as GitHub, Jenkins, and SonarQube. All others had experience with at least two out of these three platforms. The self-assessment results of the experimental group and control group were statistically not significantly different.

4.5. Data collection

The answers of the study participants were manually preprocessed for an analysis of the data with R [12].

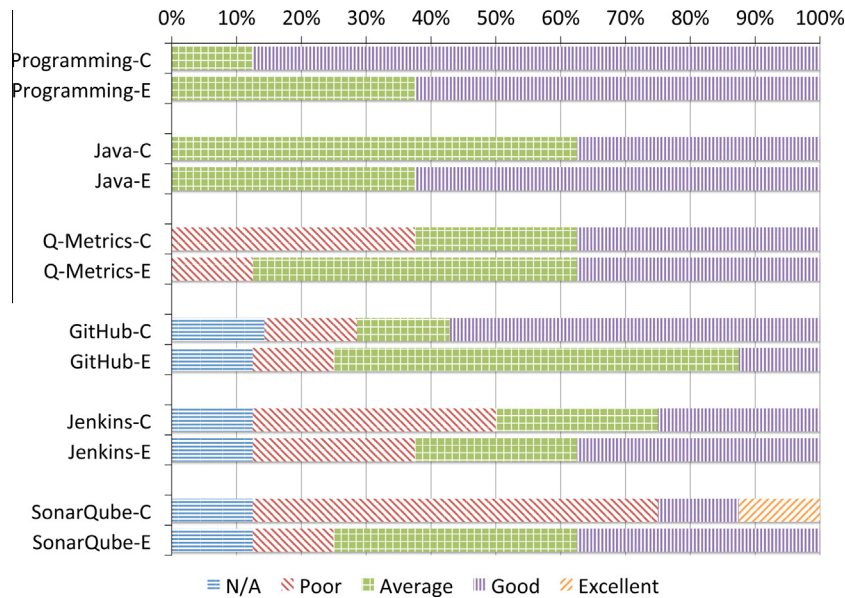


Fig. 13. Self-assessment – (C)ontrol group and (E)xperimental group.

The questionnaire of the user study consists of single choice questions, open questions, and rating scales for usability related questions. Every answer of the questionnaire was mapped to a numeric representation to run a statistical analysis on it. We used the following rules for the mapping:

- *Rating scales:* The index of the selected entry is the numerical representation, starting with zero for the first entry.
- *Single choice and open questions:* A grading scheme based on points with 3 points for a correct answer, 2 points for a partly correct answer (for open questions only), 1 point for a wrong answer, and 0 points for no answer.

We graded an answer of an open question as partly correct if it was correct but incomplete. For example, if the correct answer consists of two artifacts but only one was found by a participant. In case of an open question with only one correct answer the grading is the same as for single choice questions.

5. Empirical results

In this section we statistically examine the results of the user study with respect to the hypotheses as listed in Table 2. In Section 5.1 we analyze $H1_0$ and $H2_0$ using the aggregate results: total score per subject, total time per subject, and the ratio of total score to total time per subject aggregated over all nine tasks. In Section 5.2 we analogously analyze $H1_0$ and $H2_0$ on the level of each individual task. The results of the SUS and the difficulty ratings are discussed in Section 5.3.

5.1. Analysis & overview of aggregated results

Total Score: Table 5 lists the total score per subject aggregated over all tasks for both, the experimental and control group. Each of the nine tasks was graded with 3 points if solved correctly (see Section 4.5), so the maximum score a participant could achieve was 27. The median of the total scores was 20.0 points in the control group and 25.5 points in the experimental group. In other words, this median difference of 5.5 points means that the experimental group outperformed the control group on average by 20.4% ($=5.5/27$) when referring to the total score per subject. The minimum

Table 5

Results per subject aggregated over all tasks including median and standard deviation (SD).

Subject	Group	Score	Time (s)	Score/time * 100
S1	E	26	853.0	3.05
S2	E	26	984.0	2.64
S3	E	26	1789.0	1.45
S4	E	17	1620.0	1.05
S5	C	22	1760.0	1.25
S6	C	23	1730.0	1.33
S7	C	19	1927.0	0.99
S8	E	25	980.0	2.55
S9	E	22	1300.0	1.69
S10	C	24	1415.0	1.70
S11	C	18	2097.0	0.86
S12	C	12	2100.0	0.57
S13	E	22	1120.0	1.96
S14	E	26	841.0	3.09
S15	C	20	1740.0	1.15
S16	C	20	1940.0	1.03
Median	C	20.0	1843.5	1.09
Median	E	25.5	1052.0	2.26
SD	C	3.73	226.88	0.36
SD	E	3.24	355.06	0.76

total score per subject was 18 points (S12) and 17 points (S4), the maximum was 24 points (S10) and 26 points (S1–3,14) in the control and the experimental group, respectively. None of the participants solved all tasks correctly. One outlier (S12) with 12 points in the control group solved only four tasks - but those correctly. We could not observe any particularly obvious reasons, such as technical difficulties, tool failure, missing experience with the tools, or misunderstandings of the tasks, for this comparably low score and hence did not exclude subject S12 from our analysis.

The box-plot of the total score per subject (see Fig. 14) of the experimental group shows that the 0.75 quantile and the median are close and almost the same as the upper whisker of the plot. This indicates a skewed distribution. Furthermore, Q-Q plots and Shapiro–Wilk tests showed significant evidence against normality in the data. We made similar observations regarding the other quantities (see Figs. 14–17) although the departures are less. Since our group sample sizes are below the commonly accepted rule of

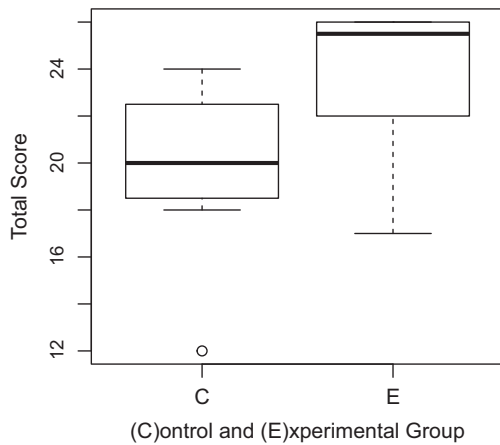


Fig. 14. Total score.

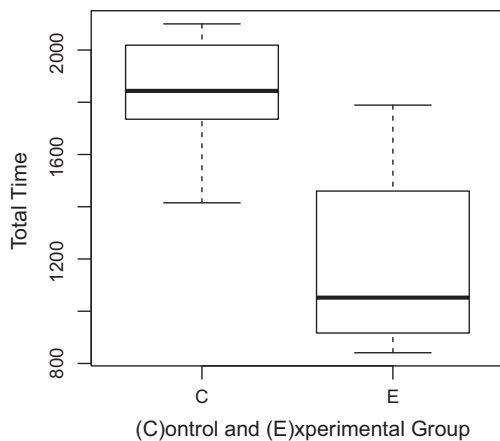


Fig. 15. Total time.

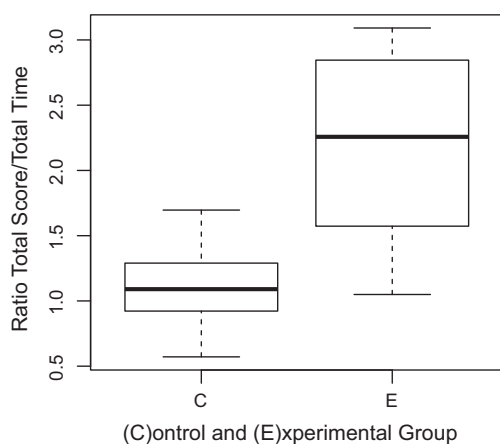


Fig. 16. Ratio total score/time.

thumb (at minimum 30 to 40) we chose a non-parametric, independent-two-samples procedure, i.e., Mann–Whitney–Wilcoxon (MWW), to test all hypotheses. Generally speaking, MWW determines if the values in the experimental group are (significantly) different by comparing the mean ranks of the values in both groups. If both groups come from identical distribution they have equal chances for high and low ranks, and there is no difference in their mean ranks. Note, that the actual test itself is based on the U statistic [13]. The box-plots further reveal that distributions of the values between the experimental and the control group might exhibit dissimilar shapes. In this case, MWW is interpreted

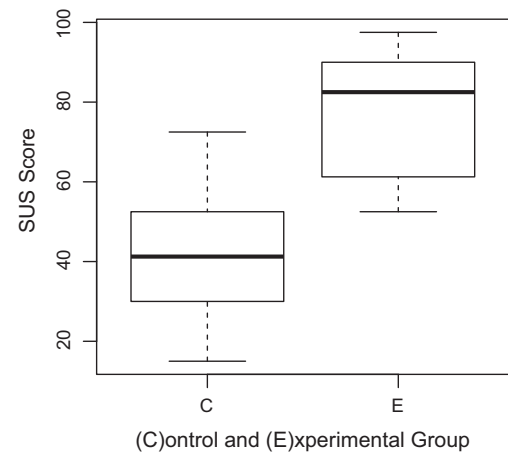


Fig. 17. SUS score.

as a test of stochastic equality [13] rather than a test of medians. Although MWW test is a non-parametric test and can handle non-normality as well as extreme data it still makes the assumption that the distributions have somewhat similar shapes and variances under the null hypothesis. Our data indicates that these assumptions might not be completely met. However, MWW test is fairly robust against deviations from the assumptions, and we consider this as a minor threat to validity.

Under the above considerations H_{10} (see Table 2) states that the mean ranks of total score per subject of the two groups are equal. A two-sided MWW test resulted in a p -value of 0.038 and 0.95 (non-parametric) confidence interval bounds of [1.7]. This gives us significant evidence against H_{10} : The total scores per subject in the experimental group (mean rank = 11) are significantly different than for the control group (mean rank = 6.0). In other words, based on the p -value and direction of the difference (as indicated by the interval bounds) we find significant evidence that the subjects of the experimental group score higher.

Total Time: Table 5 lists the total time per subject aggregated over all tasks for both, the experimental and control group. The median of the total time was 30.7 min in the control group and 17.5 min in the experimental group. This median difference of 13.2 min means that the experimental group outperformed the control group on average by 29.3% ($=13.2/45$) when referring to the total time per subject. The maximum total time per subject was 35 min (S11, S12), the minimum was 14 min (S14) and 14.2 min (S1) in the control and the experimental group, respectively. However, none of the participants reached the time limit of 45 min.

H_{20} (see Table 2) states that the mean ranks of total time per subject of the two groups are equal. A two-sided MWW test resulted in a p -value of 0.003 and (non-parametric) 0.95 confidence interval bounds of $[-977, -308]$. This gives us significant evidence against H_{20} : The total time per subjects in the experimental group (mean rank = 5.12) is significantly different than for the control group (mean rank = 11.88). Based on these results we find strong evidence that the subjects of the experimental group could solve the tasks in less time.

Ratio of Total Score to Total Time: Table 5 lists the total score to total time ratio (st-ratio) per subject aggregated over all tasks for both, the experimental and control group. This value was multiplied by 100 to aid readability. The median of the st-ratio was 1.09 in the control group and 2.26 in the experimental group. In other words, this median difference of 1.17 means that the experimental group outperformed the control group by about plus 0.7 points ($1.17/100 * 60$) per minute.

A two-sided MWW test resulted in a p -value of 0.005, (non-parametric) 0.95 confidence interval bounds of [0.363, 1.798], and mean ranks of 11.75 and 5.25, for the experimental and control group.

Table 6
Results per task.

Task	Score (median)			Time (median)		
	Ctrl.	Experm.	Δ	Ctrl.	Experm.	Δ
T1	3.0	3.0	0.0	67.5	40.0	27.5
T2	3.0	3.0	0.0	112.0	67.5	44.5
T3	1.0	2.0	1.0	285.0	225.0	60.0
T4	3.0	3.0	0.0	130.0	53.5	76.5
T5	1.0	3.0	2.0	240.0	50.0	190.0
T6	3.0	3.0	0.0	125.0	79.0	46.0
T7	0.5	2.0	1.5	300.0	300.0	0.0
T8	3.0	3.0	0.0	300.0	175.0	125.0
T9	3.0	3.0	0.0	300.0	150.0	150.0

5.2. Task analysis

The aggregated results in the previous Section 5.1 show a significant evidence that the subjects using our SQA-Mashup approach tend to solve the tasks more correctly in less time compared to subjects using the given set of baseline tools. Correctness in our context is defined as the score achieved by a participant: A higher score means a higher correctness. The goal of this section is to further investigate where the advancements come from, and we break down the analysis of the global hypotheses H1 and H2 into individual tasks.

Due to the same reasons as on the aggregate level (non-normality and sample size below 30) the analyses on each task were performed with the MWW procedure. However, because of their post hoc character we applied the Bonferroni-Holm [14] correction to the resulting p -values of the MWW tests on task level (see Table 6). This correction counteracts the problem of multiple hypotheses testing by controlling the family-wise error rate.

5.2.1. Task T1 – Learning about project size

All participants in both groups were able to find this information and solved the task correctly. Moreover, a large adjusted p -value, such as $p > 0.2$, clearly indicates that the observed time difference between the two groups is not significant.

Information about the project size is readily available in a particular view of the CI-toolchain. Hence not surprisingly, all participants of control group were able to write down the correct answer. We hypothesize that the single point of access for all information as provided by SQA-Mashup allows to spot the project size immediately and might offered the experimental group a head start. However, currently the results are not statistically conclusive and further work is needed on that issue, for instance, including additional participants.

5.2.2. Task T2 – Finding exceptionally complex source code

Every participant of both groups solved the task correctly. With respect to time a large adjusted p -value, such as $p > 0.2$, shows that the measured time difference between both group is not significant. Again, analyzing additional subjects increases power of the test and would give more confident estimates and insights whether the observed times differences of such magnitudes in this task are due to chance.

5.2.3. Task T3 – Learning about quality evolution

This task was solved with a significantly higher correctness (adjusted p -value of 0.025) by the experimental group but not necessarily faster (large adjusted p -value of $p > 0.2$).

Task T3 captures an essential activity of CI, rigorously monitoring the state and quality of a software system over time [15]. However, to facilitate a comprehensive view of the state of the system, different quality indicators, such as test coverage, code metrics,

test failures, or coding violations, must be considered. The subjects of the control group had to manually search through the individual CI-tools to locate this information. Furthermore, they faced the difficulty that each tool uses different time periods and scales to calculate and visualize information. We generally noticed that only two subjects in the control group provided a partially correct answer, the remaining six subjects answered the task wrong. We interpret the results that a consistently integrated interface can foster project visibility and helps to encourage team awareness with respect to different software quality aspects.

5.2.4. Task T4 – Learning about test coverage

14 participants solved the task correctly and only one participant out of each group gave a wrong answer. With respect to time a large adjusted p -value, such as $p > 0.2$, shows that there is no significant difference between the two groups.

5.2.5. Task T5 – Finding high execution times

This task was solved with a significantly higher correctness (adjusted p -value of 0.040) by the experimental group. We found a certain evidence (adjusted p -value of 0.073) that less time was needed.

While the experimental group likely benefited from the integrated view the subjects of the control group might have struggled with the fact that multiple CI-tools provide redundant information about execution times.

5.2.6. Task T6 – Finding test coverage below average

Only one participant was not able to solve the task correctly. Moreover, with respect to time a large adjusted p -value, such as $p > 0.2$, shows that the measured difference is not significant.

5.2.7. Task T7 – Relating build information to issues

This task was solved correctly by only two subjects of the control group and four subjects of the experimental group. Generally, this task seemed time-consuming and difficult since 75% of all participants reached the time limit of five minutes and over 50% rated it as “Very difficult” (see Fig. 18).

5.2.8. Task T8 – Learning about activities of coworkers

This task was solved significantly faster (adjusted p -value of 0.050) by the experimental group but not necessarily with higher correctness (large adjusted p -value).

The participants of the control group mentioned in their written feedback that they had difficulties in finding a specific view corresponding to the time period which we asked for in the user study. SQA-Mashup, on the other hand, offers a coherent, chronological view of developers and their activities.

5.2.9. Task T9 – Learning about activities between builds

This task was solved significantly faster (adjusted p -value of 0.033) by the experimental group but not necessarily with higher correctness (large adjusted p -value).

Task 9 is similar to task T8 from the perspective of builds. We expected a decrease in time to solve the task for both groups compared to task T8 because of learning effects and since builds are the central aspect of CI-tools. Only the median time of the experimental group decreased to 150 s, the median time of the control group remained high at 300 s. The scores increased in both groups.

5.3. Participant ratings

We asked each participant to complete the SUS questionnaire and to rate the *Difficulty* of each task (Fig. 18).

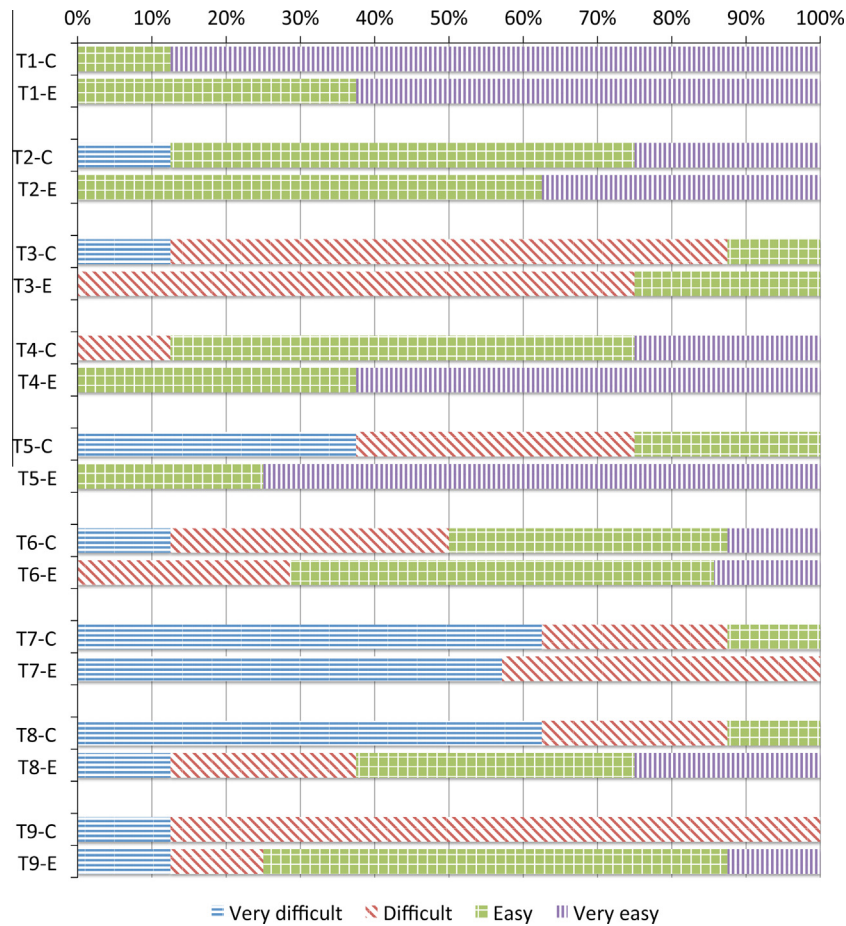


Fig. 18. Difficulty Rating – (C)ontrol group and (E)xperimental group.

The result of a SUS is a score between 0 and 100 to represent the usability of a system. In our study, the score was calculated based on the original scoring scheme of Brooke [11] as follows: The contribution of every item with an odd index equals the score minus one, the contribution of every item with an even index is calculated as five minus the score. Finally, the sum of the resulting contributions is multiplied by 2.5. Fig. 17 depicts a box-plot based on the SUS scores of the control and the experimental group. The usability median of the control group is 41.25 compared to 82.5 of the experimental group. Note, it is by chance that the usability measured in the experimental group is exactly twice of the control group's usability. The minimum whisker of the experimental group with 52.5 is slightly above the 0.75 quantile of the control group with a usability of 48.75.

H₃₀ (see Table 2) states that the mean ranks of the usability scores per subject of the both groups are equal. A two-sided MWW test resulted in a p -value of 0.004 and 0.95 confidence interval bounds of [15,55]. This gives us significant evidence against H₃₀: The usability score per subject in the experimental group (mean rank = 11.94) is significantly different from the score per subject in the control group (mean rank = 5.06). Based these findings there is strong evidence that the subjects of the experimental group perceived the integrated SQA-Mashup front-end more "user friendly".

Next, we describe differences in the individual task ratings regarding *Difficulty* between the control group and experimental group (see Fig. 18). Two-sided MWW tests on each individual task rating resulted in large p -values. Noticeable differences are the ratings of the control group on task T2, T3, T5, and T6. In all cases the

control group rated the three tasks as "Very difficult" in difference to the experimental group.

5.4. Discussion of the results

Overall we found evidence that the participants of the experimental group solved the tasks of our study in less time and with a higher correctness. The SQA-Mashup approach was rated as more "user friendly" than the given baseline tools reflected by a significantly higher SUS score.

When analyzing these differences on the level of the individual tasks we found major insights in the benefits of monitoring the CI-process and the capabilities of existing CI-tools. On the one hand, regarding single aspects of software quality, such as project size (T1) or code complexity (T2), existing CI-tools provide already good support. The participants of both groups achieved a high correctness, and we could not observe a significant time gain in either of the two groups.

On the other hand, we found evidence that monitoring software quality during CI can substantially benefit from an integrated view. This is particularly the case when the required information is scattered over multiple CI-tools and displayed in different scales and units, for instance, as it is the case for tasks T3 or T8. The subjects of the control group had to search the CI-toolchain and then manually combine the information obtained from the individual tools. This seems to be a disadvantage compared to the experimental group that was presented a single interface that integrates all relevant quality information. Therefore, we see potential for such

integrated approaches when multiple facts need to be merged to answer an aggregated quality aspect of a software system.

5.5. Threats to validity

Unequal distribution of experience in terms of development experience and experience with the used tools can bias the results of a user study. We addressed this issue by a random assignment of the participants to a group. The analysis of both experiences with the MWW test indicated no significant difference of the two populations.

The *number of participants* might have influenced the drawn conclusion. Many results had a clear tendency but they were not statistically significant. A greater number of participants might help to draw a clear conclusion for those results.

Learning effects caused by related tasks, which have to be solved within a short period of time. The correctness and completion time results may have been affected by learning effects because every group used a fixed set of tools to solve all tasks. This threat may especially apply for the tasks T7–T9. In each of these three tasks the occurrence time of an event is important for the interlinking across the CI-tools. Only the analysis of task T8 and T9 showed that score increased in both groups and almost equally.

Incorrect task completion times might have been reported by the participants. Each participant himself was responsible for tracking and writing down the time needed to solve a task. We monitored the time tracking activity of the participants during the study but it is still possible that incorrect task times have been reported.

Generalizability of the results. This threat is in respect to the participants of our study which were experienced but not professional developers. We tried to overcome this by selecting students with development experience in industry.

Authenticity of the tasks for the daily work of a software developer. The task selection of our user study is only a small extract of tasks which are done during software maintenance. We selected tasks from literature to avoid any favor in the direction of the baseline tools or SQA-Mashup.

6. Related work

The goal of our approach is to support the answering of common questions in the context of software evolution and continuous integration. We follow insights of prior research analyzing the information needs of developers to ensure that our set of questions is relevant and useful. We divide the field of the related work into the following areas: software evolution in general, information needs, continuous integration, and tool integration in Software Engineering.

Software evolution: Mens et al. [16] list a number of challenges in software evolution. One approach to reason about the evolution of software systems is *the integration of data from a wide variety of sources*. The Moose project [17] is a platform, which addresses the integration of data from different of data sources, such as source code repositories or issue trackers. FM3 is the meta-model used in Moose, which builds the base for an automatic evolution analysis. Another approach based on Semantic Web technologies was proposed by Li and Zhang [18]. Our approach tackles the integration differently since we do not use a meta-model, such as Moose, to analyze the evolution of a software system. Instead, we integrate data with what is already computed by state-of-the-art CI-tools.

Information needs: Aranda and Venolia [7] investigated source code histories in repositories to identify common bug fixing patterns. Breu et al. [19] extracted a catalog of questions from bug reports. They found that many questions are not explicitly stated in issues and therefore often not answered. Other studies conclude with an explicit catalog of questions asked by a group of stakeholders. LaToza and Myers [6] came up with a list of *Hard-to-answer*

questions about code. Fritz and Murphy [2] provide a collection of developer's questions in combination with an information fragment model to answer these questions. Another group of studies examined the tasks of professionals during software comprehension. Roehm et al. [20] showed that developers follow different strategies to complete their work. Müller and Fritz [21] shifted the focus from software testers and developers to a more diverse audience such as requirements engineers and product line managers. They found that these stakeholders require multiple different artifacts to perform their daily activities. Their findings extend existing studies [2,6] performed with software developers.

Continuous integration: Staff and Ernst reported on a controlled human experiment to evaluate the impact of continuous testing onto the source code quality [22]. The results of their experiment showed an improvement caused by the immediate feedback from the continuous testing framework. Their approach was implemented as plugins for Eclipse and Emacs. A similar experiment was conducted by Hurdugaci and Zaidman [23]. They implemented a plugin for Visual Studio, which *helps developers to identify the unit tests that need to be altered and executed after a code change* [23]. Both experiments showed that an immediate feedback from CI-tools fosters software quality during software development and maintenance. We base upon these results and aim at mashing up the proper information for the respective stakeholder.

Tool integration in software engineering: Wasserman [24] described various types of tool integration to show their potential for computer-aided software engineering (CASE) environments. The missing standards for tool integration in CASE restrict the integration of data mainly on a tool-to-tool level. Michaud et al. [25] proposed an approach to support the exploration of source code and documentation of Java programs. They used existing standalone tools to extracted data from source code or documentation and integrated for a visualization based on the SHrIMP technique [26].

7. Conclusions

The establishment of a CI-toolchain can prevent a software project from descending into *Continuous Integration Hell*, but the effort to overlook the amount of data generated by CI-tools in a toolchain increases with each additional tool.

We introduced a mashup and visualization framework for CI-data called SQA-Mashup, which tailors the data according to the information needs of stakeholders, such as software developer and testers. SQA-Mashup implements a pipe-and-filter based data integration approach and a role-based visualization approach built on a dynamic view concept. We derived information needs and tasks from related works for the design and the evaluation of our proof-of-concept implementation.

We showed that the SQA-Mashup approach can (1) foster visibility of the project status, (2) improve team awareness, and (3) help spreading consistent quality information. A controlled user study with 16 participants was conducted to support this claim. In summary, the results of our controlled user study provide evidence for the following:

- An integrated view on CI-data that effectively combines multiple CI-tools can positively influence the monitoring of software quality within a software project. The participants of our study, which used SQA-Mashup achieved a 21.6% higher correctness when answering questions about the evolution and quality of a software project than those using standalone CI-tools.
- Our integrated approach leads to significant time savings in monitoring software quality data compared to the use of standalone CI-tools. The tasks were solved 57% faster with SQA-Mashup compared to standalone CI-tools.

- The overall user satisfaction based on the System Usability Score with our integrated approach is significantly better compared to the user satisfaction with the standalone CI-tools used in the control group of our study.

Our approach serves as *one-stop shop* for software quality data monitoring in a software development project. It enables easy access to CI-data which otherwise is not integrated but scattered across multiple CI-tools. Our dynamic visualization approach allows for a tailoring of integrated CI-data according to information needs of different stakeholders such as developers or testers.

The prototypical implementation of SQA-Mashup is available in the Windows 8 App Store and a walk-through can be found on our Web-site.⁵

Future work will focus on further visualizations of integrated CI-data, especially for time-related information (e.g., commit and build events visualized on a timeline). We also plan to enable a search for reoccurring patterns, for example, to find more than *n* commits within some minutes before a build failure.

References

- [1] M. Fowler, Continuous integration. <<http://www.martinfowler.com/articles/continuousIntegration.html>> (Last accessed: 09/01/2014).
- [2] T. Fritz, G.C. Murphy, Using information fragments to answer the questions developers ask, ICSE, Cape Town, 2010, pp. 175–184. <http://dx.doi.org/10.1145/1806799.1806828>.
- [3] J. Singer, T. Lethbridge, N. Vinson, N. Anquetil, An examination of software engineering work practices, CASCON, Toronto, 1997, pp. 21–36. doi:10.1.1.40.705.
- [4] M. Brandtner, E. Giger, H. Gall, Supporting continuous integration by mashing-up software quality information, CSMR-WCRE, 2014, pp. 184–193. <http://dx.doi.org/10.1109/CSMR-WCRE.2014.6747169>.
- [5] A. Mockus, J.D. Herbsleb, Expertise browser: a quantitative approach to identifying expertise, ICSE, Orlando, 2002, pp. 503–512. <http://dx.doi.org/10.1145/581339.581401>.
- [6] T.D. LaToza, B.A. Myers, Hard-to-answer questions about code, in: Evaluation and Usability of Programming Languages and Tools, PLATEAU, Reno, 2010, pp. 8:1–8:6. <http://dx.doi.org/10.1145/1937117.1937125>.
- [7] J. Aranda, G. Venolia, The secret life of bugs: going past the errors and omissions in software repositories, ICSE, Washington, 2009, pp. 298–308. <http://dx.doi.org/10.1109/ICSE.2009.5070530>.
- [8] A. Mockus, N. Nagappan, T.T. Dinh-Trong, Test coverage and post-verification defects: a multiple case study, ESEM, Washington, 2009, pp. 291–301. <http://dx.doi.org/10.1109/ESEM.2009.5315981>.
- [9] R. Wettel, M. Lanza, R. Robbes, Software systems as cities: a controlled experiment, ICSE, Waikiki, 2011, pp. 551–560. <http://dx.doi.org/10.1145/1985793.1985868>.
- [10] P. Louridas, Junit: unit testing and coiling in tandem, Software, IEEE 22 (4) (2005) 12–15. doi:10.1109/MS.2005.100.
- [11] J. Brooke, Sus-a quick and dirty usability scale, Usab. Eval. Indus. 189 (1996) 194.
- [12] R Development Core Team, R: A Language and Environment for Statistical Computing, Vienna, 2011.
- [13] H. Mann, D. Whitney, On a test of whether one of two random variables is stochastically larger than the other, Ann. Math. Stat. 18 (1) (1947) 50–60.
- [14] S. Holm, A simple sequentially rejective multiple test procedure, Scand. J. Stat. 6 (2) (1979) 65–70. doi:10.2307/4615733.
- [15] P.M. Duvall, S. Matyas, A. Glover, Continuous Integration: Improving Software Quality and Reducing Risk, Addison-Wesley, 2007.
- [16] T. Mens, M. Wermelinger, S. Ducasse, S. Demeyer, R. Hirschfeld, M. Jazayeri, Challenges in software evolution, IWPSE, Lisbon, 2005, pp. 13–22. <http://dx.doi.org/10.1109/IWPSE.2005.7>.
- [17] O. Nierstrasz, S. Ducasse, T. Girba, The story of moose: an agile reengineering environment, SIGSOFT Softw. Eng. Notes 30 (5) (2005) 1–10. doi:10.1145/1095430.1081707.
- [18] Y.-F. Li, H. Zhang, Integrating software engineering data using semantic web technologies, MSR, Waikiki, 2011, pp. 211–214. <http://dx.doi.org/10.1145/1985441.1985473>.
- [19] S. Breu, R. Premraj, J. Sillito, T. Zimmermann, Information needs in bug reports: improving cooperation between developers and users, CSCW, Savannah, 2010, pp. 301–310. <http://dx.doi.org/10.1145/1718918.1718973>.
- [20] T. Roehm, R. Tiarks, R. Koschke, W. Maalej, How do professional developers comprehend software? ICSE, Piscataway, 2012, pp. 255–265. <http://dx.doi.org/10.1109/icse.2012.6227188>.
- [21] S.C. Müller, T. Fritz, Stakeholders' information needs for artifacts and their dependencies in a real world context, ICSM, Eindhoven, 2013. <http://dx.doi.org/10.1109/ICSM.2013.40>.
- [22] D. Saff, M.D. Ernst, An experimental evaluation of continuous testing during development, SIGSOFT Softw. Eng. Notes 29 (4) (2004) 76–85. doi:10.1145/1013886.1007523.
- [23] V. Hurdugaci, A. Zaidman, Aiding software developers to maintain developer tests, CSMR, Szeged, 2012, pp. 11–20. <http://dx.doi.org/10.1109/CSMR.2012.12>.
- [24] A. Wasserman, Tool integration in software engineering environments, in: Software Engineering Environments, Lecture Notes in Computer Science 467, 1990, pp. 137–149.
- [25] J. Michaud, M. Storey, H. Muller, Integrating information sources for visualizing java programs, ICSM, 2001, pp. 250–258. doi:10.1.1.24.8515.
- [26] M.A.D. Storey, K. Wong, F. Fracchia, H. Muller, On integrating visualization techniques for effective software exploration, InfoVis, 1997, pp. 38–45. doi:10.1.1.25.9705.

⁵ <http://www.ifi.uzh.ch/seal/research/tools/sqa-mashup>.